

FICHA 1 – JAVASCRIPT E THREE.JS

OBJETIVOS:

- Conhecer a sintaxe básica de JavaScript, a sua integração em páginas Web e utilização em aplicações Web.
- Conhecer a utilização de JavaScript na implementação de interatividade e animação.
- Conhecer os componentes essenciais de projeto 3D usando a biblioteca Three.js.
- Conhecer os utilitários AxesHelper, GridHelper, OrbitControls e Stats.
- Importar em Three.js uma cena criada numa ferramenta de modelação tridimensional.



ACONSELHA-SE QUE A RESOLUÇÃO DE TODOS OS EXERCÍCIOS PROPOSTOS SEJA FEITA INDIVIDUALMENTE.

Parte I. PREPARAÇÃO DO AMBIENTE DE DESENVOLVIMENTO

Antes de avançar deve ter preparado o ambiente de desenvolvimento!

Para tal deve averiguar as 3 primeiras secções do “Guião de Preparação do Ambiente de Desenvolvimento” e ter o projeto exemplo, de nome “sample project”, funcional.

Para a realização dos exercícios relativos à biblioteca *ThreeJS*, deverá criar uma pasta por exercício dentro da pasta “ThreeJS Projects”.

Recomenda-se a leitura das restantes secções do guião após a conclusão desta ficha.

Parte II. JAVASCRIPT

A linguagem JavaScript é uma linguagem de programação interpretada suportada nativamente por todos os clientes Web atuais. Forma, em conjunto com as linguagens HTML e CSS, o triângulo de tecnologias de base da produção de conteúdo Web. A linguagem JavaScript permite, entre outras coisas: manipular HTML e CSS; responder em tempo real à interação do utilizador (ex.: cliques ou movimentos de rato); e criar automatismos do lado do cliente (ex.: animações, validações, etc.).

Para incluir código JavaScript num documento HTML geralmente é utilizado o elemento HTML `<script>`. Este elemento pode ser usado de duas formas:

- Executando código JavaScript incluído no próprio ficheiro HTML

```
<script>
// código JavaScript
</script>
```

- Importando código JavaScript a partir de um ficheiro externo (usualmente com extensão “.js”)

```
<script src="ficheiro.js"></script>
```

Um mesmo ficheiro HTML pode conter múltiplos elementos `<script>` e estes serão executados pela ordem que aparecem no ficheiro HTML. Os elementos script podem aparecer em qualquer sítio dentro dos elementos `<head>` e `<body>`.

Em termos de sintaxe, a linguagem JavaScript é bastante similar a linguagens como C, C++ e Java, partilhando com estas muitos dos operadores (nomeadamente os de comentário) e estruturas de controlo. Contudo, há que referir as seguintes particularidades:

- É uma linguagem interpretada (não é necessário compilar antes de executar);
- A utilização do ponto e vírgula no final das instruções é opcional quando há uma mudança de linha;
- As funções são objetos;
- Variáveis não declaradas são criadas implicitamente na sua primeira utilização;
- Usa tipos dinâmicos (o tipo de uma variável depende do valor que ela contém no momento).

Tal como muitas outras linguagens baseadas em objetos, permite manipular propriedades de um objeto usando o operador ponto.

```
obj.propriedade = valor
```

Onde **obj** é uma variável que refere um objeto, **propriedade** é o nome de uma propriedade do objeto e **valor** é o valor que será copiado.

É igualmente possível usar o operador ponto para invocar métodos de um objeto:

```
obj.metodo()
```

Onde **obj** é uma variável que refere um objeto, **metodo** é o nome de um método desse objeto.

Propriedades e métodos são objetos, pelo que o operador ponto pode ser usado múltiplas vezes na mesma expressão. Exemplos:

```
obj.propriedade.metodo()
obj.propriedade1.propriedade2
obj.metodo().propriedade
```

1. TRATAMENTO DE EVENTOS

Considere o ficheiro fornecido **contentor-botoes.html**. Observe os três elementos HTML identificados, isto é, com valor atribuído ao seu atributo **id**.

- 1.1. Dentro do próprio ficheiro HTML crie um elemento script, declare três variáveis, utilize o método **document.getElementById()** para obter referências aos três elementos HTML identificados, e atribua estas referências às variáveis previamente declaradas.

Para declarar a variável **x** utilizamos a expressão:

```
let x
```

A sintaxe do método **document.getElementById** é:

```
document.getElementById('id_x')
```

Onde **id_x** é o identificador (valor do atributo **id**) do elemento HTML que queremos referenciar.

- 1.2. Crie as funções **mudarParaCinza** e **mudarParaVerde**.

A sintaxe da declaração de funções em JavaScript é a seguinte:

```
function nomeFuncao() {  
    // código a executar...  
}
```

Ou, em alternativa, usar uma expressão de função (função sem nome) e atribuir a uma variável:

```
let nomeFuncao = function() {  
    // código a executar...  
}
```

- 1.3. A cada uma das funções criadas no ponto anterior, utilize a propriedade **style** do contentor para alterar a sua cor de fundo para o valor apropriado.

A sintaxe desta expressão é seguinte:

```
obj.style.propriedadeCSS = valorCSS
```

Onde **obj** é uma referência para o objeto HTML, **propriedadeCSS** é o nome da propriedade CSS e **valorCSS** é o valor a atribuir à propriedade. Tenha em atenção que, no contexto JavaScript, os valores de propriedades CSS geralmente são cadeias de caracteres (ex.: **"10px"**, **"green"**), e que os nomes das propriedades compostas por duas ou mais palavras usam a convenção da capitalização e não a da hifenização (isto é, em CSS escreve-se **font-size** e em JavaScript, **fontSize**).

- 1.4. Utilize o método **addEventListener()** para associar as funções criadas nos pontos anteriores ao tratamento do evento clique do botão de rato.

A sintaxe deste método é:

```
obj.addEventListener('evento', nomeFuncao)
```

Onde **obj** é o objeto sobre o qual o evento vai ser capturado, **evento** é o nome do evento a capturar (ex.: **click**, **mousemove**, **mousedown**, **mouseup**), e **nomeFuncao** é o nome da função que irá tratar esse evento.

Em alternativa, usar a seguinte expressão:

```
obj.onevento = nomeFuncao
```

- 1.5. Como as funções só são referidas uma vez, podemos dispensar o uso de nomes e usar diretamente uma expressão de função. Por exemplo:

```
obj.addEventListener('evento', function() {  
    // código a executar  
})
```

ou

```
obj.onevento = function() {  
    // código a executar  
}
```

2. ANIMAÇÃO

Para implementar uma animação básica em JavaScript usa-se o método `requestAnimationFrame()`. Este método recebe como parâmetro a função que será invocada no próximo quadro, ou *frame*, da animação. Essa função tem como estrutura básica:

```
function animar() {  
    requestAnimationFrame(animar)  
    atualizar()  
    mostrar()  
}
```

Onde `mostrar()` será uma função que se encarrega de fazer a visualização da *frame* (ex.: desenhar o estado de uma cena) e `atualizar()` efetuará as alterações necessárias para a próxima *frame* (ex.: alterar de posição objetos de uma cena).

- 2.1. Modifique o ficheiro `contador.html` para que o número mostrado avance uma unidade em cada *frame*. O botão “Iniciar” inicia a animação, e o botão “Parar”, para-a.

Sugestões: Utilize uma variável global para armazenar o valor do contador. Para mostrar o contador na página, utilize a propriedade `innerHTML` do elemento `<span>`. Utilize uma variável global auxiliar para parar a animação.

Parte III. THREE.JS: INTRODUÇÃO

Originalmente, os clientes web não tinham suporte nativo para gráficos 3D e era necessário recorrer a *plugins*, com os múltiplos inconvenientes que isso tem (sobretudo falta de compatibilidade e falhas de segurança). Isso mudou com a introdução da API WebGL. WebGL é uma API de JavaScript baseada em OpenGL e que desenha dentro de um elemento HTML5 `<canvas>`. Atualmente, praticamente todos os clientes web têm suporte para WebGL (ver: <http://caniuse.com/#feat=webgl> e <http://caniuse.com/#feat=webgl2>). Contudo, o seu suporte também depende do hardware (GPU).

Exemplos de WebGL 2.0 em utilização: [Messenger](#), [Body](#), [Car visualizer](#), [Jellyfish](#).

WebGL é uma API de *baixa nível*, pelo que requer um esforço elevado de programação para criar cenas 3D complexas. Existem diversas bibliotecas JavaScript de *alto nível* que permitem esconder muita desta complexidade. Nesta UC será utilizada especificamente a biblioteca Three.js.

Com a introdução dos ES Modules na especificação de JavaScript, presentemente suportados pela maioria dos navegadores web (ver: <https://caniuse.com/es6-module>), a biblioteca Three.js está a transacionar para um modo de desenvolvimento baseado em módulos. Para além de resolver inconveniências relativas à ordem de importação dos scripts, a utilização de módulos permite um controlo mais granular das funcionalidades a incluir num projeto podendo-se exportar apenas as usadas em *bundles* ao invés de incluir bibliotecas inteiras. Ainda que a utilização de *bundles* não seja relevante no contexto de exercícios a resolver em aula, serão utilizados módulos uma vez que a biblioteca Three.js está vocacionada para estes.

Para facilitar a inclusão do módulo principal da biblioteca Three.js, assim como módulos complementares, deverá incluir os seguintes *aliases* (**importmap**) no ficheiro HTML. O excerto seguinte deve ser copiado para o final do corpo (*body*) do ficheiro html.

```
<script type="importmap">
{
  "imports": {
    "three": "/node_modules/three/build/three.module.js",
    "three/addons/": "/node_modules/three/examples/jsm/"
  }
}
</script>
```

Seguidamente, deverá ser incluído o ficheiro JavaScript com a **tag module**.

```
<script type="module" src="./main.js"></script>
```

Com isto tudo, o ficheiro HTML deverá ficar similar ao exemplo abaixo.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Three.js HTML Template</title>
</head>
<body>

<script type="importmap">
{
  "imports": {
    "three": "/node_modules/three/build/three.module.js",
    "three/addons/": "/node_modules/three/examples/jsm/"
  }
}
</script>
<script type="module" src="./main.js"></script>
</body>
</html>
```

Neste exemplo, o ficheiro `main.js` irá importar a biblioteca Three e conter o código do projeto.

A importação de todos os componentes principais da biblioteca Three deve ser feita no início do ficheiro `main.js` com a seguinte instrução:

```
import * as THREE from 'three';
```

Um projeto Three.js é constituído essencialmente pelos seguintes objetos:

- **Malhas (Mesh):** Objetos a visualizar. Por sua vez, cada um destes objetos é composto por geometria (vértices, arestas e faces) e por material (características de cor da superfície das faces).
- **Cena (Scene):** onde são dispostas as malhas.
- **Câmara (Camera):** o ponto de vista usado para fazer a projeção.
- **Renderer:** O programa que se encarrega de fazer a projeção da cena usando a informação da camara.

Cada um destes objetos é criado usando um construtor do módulo THREE, da seguinte forma:

```
let obj = new THREE.Construtor()
```

Onde `obj` é o nome da variável que irá receber a referência do objeto e `Construtor()` é o método construtor apropriado.

3. CRIAR UM CUBO 1x1x1

Uma forma simples de criar um cubo 1x1x1 é recorrer ao construtor `BoxGeometry()` que gera a geometria de um paralelepípedo dadas as suas três dimensões:

```
let geometria = new THREE.BoxGeometry( 1, 1, 1 )
```

Na secção “Geometries” da documentação de Three.js podemos ver outros construtores deste género disponíveis na API.

Como material iremos usar `MeshNormalMaterial()` que é um material simples que realça a geometria (atribui cores consoante os vetores normais às superfícies):

```
let material = new THREE.MeshNormalMaterial()
```

Para ver outros materiais disponíveis, consultar a secção “Materials” da documentação de Three.js.

Finalmente, criamos o objeto usando o método `Mesh()`:

```
let cubo = new THREE.Mesh( geometria, material )
```

4. CRIAR A CENA E COLOCAR NELA O CUBO

A cena é criada usando o construtor `Scene()`.

```
let cena = new THREE.Scene()
```

Uma vez criada a cena, podemos usar o seu método `add()` para acrescentar objetos:

```
cena.add( cubo )
```

5. CRIAR E CONFIGURAR O RENDERER

Iremos usar um renderer WebGL, pelo que o renderer será criado com o construtor `WebGLRenderer()`.

```
let renderer = new THREE.WebGLRenderer()
```

Uma vez criado o renderer, usamos o seu método `setSize()` para dimensionar o tamanho *viewport* pretendido. Neste exemplo usaremos um viewport 800x600.

```
renderer.setSize( 800, 600 )
```

Finalmente, como ainda não temos um elemento `<canvas>` no HTML, podemos acrescentá-lo dentro do elemento `<body>` usando o seu método `appendChild()`:

```
document.body.appendChild( renderer.domElement )
```

6. CRIAR E POSICIONAR A CÂMARA

A biblioteca Three.js fornece diversos tipos de câmara, cada um com o seu tipo de projeção. Nomeadamente, podemos escolher entre a projeção “perspetiva” (a mais natural e que nos dá a noção de tridimensionalidade, onde a dimensão aparente dos objetos é inversamente proporcional com a distância) e a projeção “ortogonal” (as dimensões dos objetos não variam com a distância). Neste caso iremos usar a projeção em perspetiva:

```
let camara = new THREE.PerspectiveCamera( 70, 800 / 600, 0.01, 1000 )
```

O primeiro parâmetro é o ângulo de visão da câmara.

O segundo parâmetro é a razão de aspeto da imagem projetada, que deve coincidir com a do viewport definido. Como o viewport é de 800x600, a razão de aspeto é 800:600 (ou 4:3).

O terceiro parâmetro é o “near clipping”. Isto é, o que estiver mais próximo do que este limite não é renderizado.

O quarto parâmetro é o “far clipping”. Isto é, o que estiver mais longe do que este limite não é renderizado.

Por omissão a câmara está posicionada no centro da cena, que é o mesmo sítio onde se encontra o nosso cubo. Por isso vamos afastá-la um pouco no eixo Z:

```
camara.position.z = 5
```

7. RENDERIZAR E ANIMAR

Para renderizar uma cena estática, bastaria invocar o método `render()` do `renderer`:

```
renderer.render( cena, camara )
```

Contudo, cenas estáticas não são muito interessantes pois não permitem qualquer interatividade ou animação. Por isso normalmente é usada uma função de animação semelhante à do exercício “Contador Animado”:

```
function animar() {  
    requestAnimationFrame( animar )  
  
    // atualizar rotação do cubo  
    cubo.rotateX(0.01)  
    cubo.rotateY(0.02)  
  
    // mostrar...  
    renderer.render( cena, camara )  
}
```

Neste exemplo, a atualização de cada *frame* envolve uma rotação em X e em Y do cubo. O parâmetro é um ângulo de rotação em radianos (radianos = 180 graus).

Por omissão, a regra geral é que o `requestAnimationFrame` será chamado com a taxa de atualização do monitor. De modo a evitar que uma elevada taxa de atualização cause problemas de performance, a taxa de atualização (não de invocações da função `animar`!) pode ser limitada recorrendo ao relógio da biblioteca Three.js.

A fim de limitar o número de atualizações por segundo, a seguinte modificação define a latência mínima entre atualizações com base na taxa de atualizações desejada (60 no excerto seguinte).

```
let delta = 0;           // tempo desde a última frame
let relogio = new THREE.Timer(); // componente que obtém o delta
let latencia_minima = 1 / 60; // tempo mínimo entre cada frame

function animar() {
    requestAnimationFrame(animar); // agendar animar para o próximo animation frame
    relogio.update();               // atualizar o tempo
    delta += relogio.getDelta();    // acumula tempo que passou desde a última chamada de getDelta

    // não exceder a taxa de atualizações definida
    if (delta < latencia_minima) return;

    // atualizar delta com o excedente
    delta = delta % latencia_minima;

    // atualizar rotação do cubo
    cubo.rotateX(0.01)
    cubo.rotateY(0.02)

    // mostrar...
    renderer.render( cena, camara )
}
```

De notar que as atualizações realizadas ao cubo pressupõem que a taxa de atualizações não é variável, o que pode não se verificar na prática.

Em resumo, o código no ficheiro `main.js` fica como se segue:

```
1 import * as THREE from 'three';
2
3 // Criar um cubo 1x1x1
4 let geometria = new THREE.BoxGeometry(1, 1, 1);
5 let material = new THREE.MeshNormalMaterial();
6 let cubo = new THREE.Mesh(geometria, material);
7
8 // Criar a cena e colocar nela o cubo
9 let cena = new THREE.Scene()
10 cena.add( cubo )
11
12 // Criar e configurar o renderer
13 let renderer = new THREE.WebGLRenderer()
14 renderer.setSize( 800, 600 )
15 document.body.appendChild( renderer.domElement )
16
17 // Criar e posicionar a camara
18 let camara = new THREE.PerspectiveCamera( 70, 800 / 600, 0.01, 1000 )
19 camara.position.z = 5
20
21 // Renderizar e animar
22 let delta = 0; // tempo desde a última atualização
23 let relógio = new THREE.Timer(); // componente que obtém o delta
24 let latencia_minima = 1 / 60; // tempo mínimo entre cada atualização
25
26 function animar() {
27     requestAnimationFrame(animar); // agendar animar para o próximo animation frame
28     relógio.update(); // atualizar o tempo
29     delta += relógio.getDelta(); // acumula tempo entre chamadas de getDelta
30
31     if (delta < latencia_minima) return; // não exceder a taxa de atualização
32
33     // atualizar rotação do cubo
34     cubo.rotateX(0.01)
35     cubo.rotateY(0.02)
36
37     // mostrar...
38     renderer.render( cena, camara )
39
40     delta = delta % latencia_minima; // atualizar delta com o excedente
41 }
42
43 // iniciar animar
44 animar()
```

Parte IV. AJUDAS E CONTROLOS

É de salientar que o Three.js, ao contrário de alguma ferramenta de modelação tridimensional, não possui um ambiente de desenvolvimento em 3D próprio: o que vemos será sempre o resultado da renderização de uma cena e quaisquer “vistas” na fase de desenvolvimento têm de ser implementadas através de uma câmara definida no projeto.

Por isso, embora estas ferramentas sejam úteis na fase de desenvolvimento do projeto, é provável que mais tarde tenham de ser desativadas ou removidas quando se passe à fase de produção.

8. CENA INICIAL

Criar um projeto Three.JS que contenha uma cena, uma câmara, um renderer e renderizar uma vez:

```
let cena = new THREE.Scene()

let camara = new THREE.PerspectiveCamera( 70, 800 / 600, 0.1, 500 )

let renderer = new THREE.WebGLRenderer()
renderer.setSize( 800, 600 )
document.body.appendChild( renderer.domElement )

renderer.render( cena, camara )
```

Esta cena resulta num retângulo preto visto que não tem qualquer conteúdo.

9. EIXOS DE REFERÊNCIA

Antes de renderizar a cena, crie e adicione um elemento com a referência dos eixos:

```
let eixos = new THREE.AxesHelper()
cena.add( eixos )
```

A cor permite distinguir o eixo: vermelho para X, verde para Y, azul para Z.

Teste diversas posições da câmara alterando os valores da sua posição e orientação.

Por exemplo:

```
camara.position.x = 2
camara.position.y = 3
camara.position.z = 5
// ou camara.position.set( 2, 3, 5 )
camara.lookAt( 0, 0, 0 )
```

10. GRELHA DE REFERÊNCIA

Adicione uma grelha de referência (“chão”):

```
let grelha = new THREE.GridHelper()  
cena.add( grelha )
```

11. CONTROLOS DE NAVEGAÇÃO

Dentro da diretoria `node_modules/three/examples/jsm` pode encontrar scripts com funcionalidades diversas.

Importe o componente `OrbitControls.js` no topo do ficheiro JavaScript:

```
import { OrbitControls } from 'three/addons/controls/OrbitControls.js'
```

Na instrução acima são indicados os componentes do módulo `OrbitControls.js` que se pretendem importar. Caso houvesse outros componentes a importar, poderiam ser enumerados utilizando vírgula como carácter separador.

Seguidamente acrescente os controlos orbitais:

```
let controlos = new OrbitControls( camara, renderer.domElement )
```

Para que as alterações tenham efeitos visíveis existem duas abordagens, consoante a cena seja estática ou animada.

- Se a cena for estática, podemos simplesmente renderizar quando houver alterações detetadas nos controlos:

```
function renderizar() {  
    renderer.render( cena, camara )  
}  
renderizar()  
controlos.addEventListener( 'change', renderizar )
```

- Se a cena tiver animação, como vai acontecer com a esmagadora maioria dos exemplos a implementar, o próprio ciclo de animação encarrega-se de renderizar as alterações:

```
function animar() {  
    requestAnimationFrame( animar )  
    renderer.render( cena, camara )  
}  
animar()
```

Nota: Tal como é referido no texto acima, apenas um destes excertos de código deve ser utilizado no ficheiro Javascript.

12. VISUALIZAR ESTATÍSTICAS

Idealmente, para que uma interface seja percebida como “fluida”, esta deve ser renderizada à velocidade máxima suportada pelo monitor (tipicamente 60 vezes por segundo: 60 fps ou 16ms por frame). A ferramenta Stats.js apresenta esta informação em tempo real para ajudar a identificar cenas demasiado complexas ou pesadas.

Importe o componente Stats, do módulo stats.module.js no topo do ficheiro JavaScript.

```
import Stats from 'three/addons/libs/stats.module.js'
```

Repare que a importação não utiliza chavetas, como visto anteriormente para o OrbitControls. Isto é porque Stats é o *export default* do módulo stats.module.js, e não necessita ser especificado. Em rigor, Stats é um alias para a função dentro do nosso ficheiro JavaScript, enquanto OrbitControls refere-se a um componente específico do módulo OrbitControl.js que se pretende importar.

No JavaScript acrescente:

```
let stats = new Stats()
document.body.appendChild( stats.dom )
```

Inclua na função de animação uma chamada ao método **update()**:

```
function animar() {
    requestAnimationFrame( animar )
    stats.update()
    renderer.render( cena, camara )
}
```

Parte V. IMPORTAR FICHEIRO 3D PARA THREE.JS

Para transferir cenas criadas em numa ferramenta de modelação tridimensional, como por exemplo o Blender, para Three.js é necessário um formato de ficheiro intermédio, visto que os ficheiros nativos dessas ferramentas (**.blend** no caso do Blender) não são diretamente suportados em Three.js. Por norma, essas ferramentas de modelação tridimensional têm um exportador (ver opções no menu File, opção Export – Figura 1) que produza um dos formatos para o qual exista suporte no Three.js (ex.: Collada, FBX). E do lado do Three.js é necessário um *loader* que permita interpretar o formato de ficheiro em questão e convertê-lo em objetos que podem ser adicionados a uma cena Three.js.

Nesta UC iremos utilizar o formato de ficheiro glTF (ver: <https://www.khronos.org/glTF/>). Na FIGURA 1 identifica o local onde se encontra o exportador na ferramenta Blender.

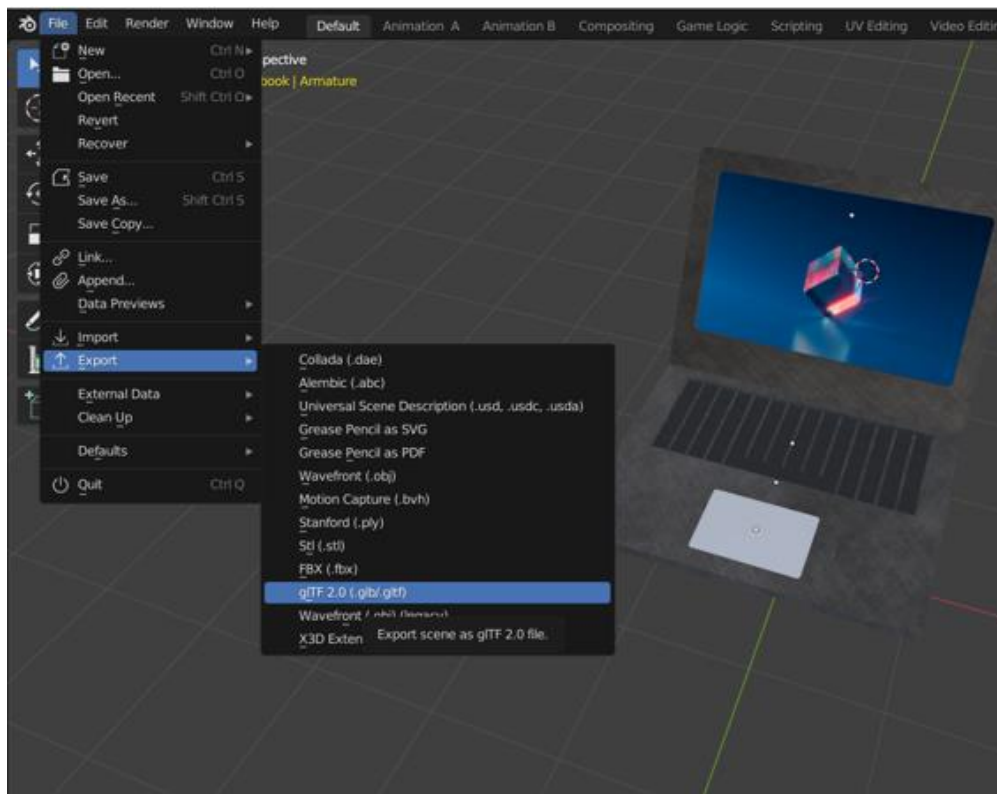


FIGURA 1. EXEMPLO DE COMO EXPORTAR PARA GLTF/GLB NO BLENDER

Se estiver a usar a extensão para visualização de modelos 3D no formato *glTF* no Visual Studio Code, certifique-se que exporta o modelo no formato *gltf* e não no formato *glb*.

O Three.js também não inclui de origem o *loader* necessário para os ficheiros glTF. É necessário importar o componente **GLTFLoader**.

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js'
```

Um *loader* de glTF é invocado da seguinte forma:

```
let carregador = new GLTFLoader()
carregador.load(
  'caminho/ficheiro.glTF',
  function ( gltf ) {
    cena.add( gltf.scene )
  }
)
```

Onde **cena** é o nome da variável que contém a cena Three.js e **ficheiro.glTF** é o ficheiro que foi exportado na ferramenta de modelação tridimensional.

13. ADICIONAR MODELO IMPORTADO DE FERRAMENTA TRIDIMENSIONAL

Crie uma cena Three.js que importe e apresente o modelo glTF disponibilizado no material de apoio (**notebook.glTF**).

14. ILUMINAÇÃO

O objeto importado estará completamente escuro. Isto deve-se ao facto de a cena não ter qualquer iluminação. Até aqui, todos os objetos adicionados não necessitavam de iluminação devido ao seu material.

Crie e adicione à cena uma fonte de luz ambiente:

```
const luzAmbiente = new THREE.AmbientLight( "lightgreen" )
cena.add(luzAmbiente)
```

Crie, posicione e adicione um ponto de luz:

```
const luzPonto = new THREE.PointLight( "red" )
luzPonto.position.set( 0, 2, 2 )
luzPonto.intensity= 15
cena.add( luzPonto )
// auxiliar visual
const LightHelper1 = new THREE.PointLightHelper( luzPonto, 0.2 )
cena.add( LightHelper1 )
```

Crie, e oriente uma luz direcional:

```
const luzDirecional = new THREE.DirectionalLight( "white" );
luzDirecional.position.set( 3, 2, 0 ); //aponta na direção de (0, 0, 0)
luzDirecional.intensity= 30
cena.add( luzDirecional );
// auxiliar visual
const LightHelper2 = new THREE.DirectionalLightHelper( luzDirecional, 0.2 )
cena.add( LightHelper2 )
```

Note que a direção da luz é dada em função da propriedade *position* (posição), sendo o seu alvo, por omissão o ponto (0, 0, 0). Assim sendo, o vetor de direção da luz é equivalente ao da sua posição multiplicado por (-1).